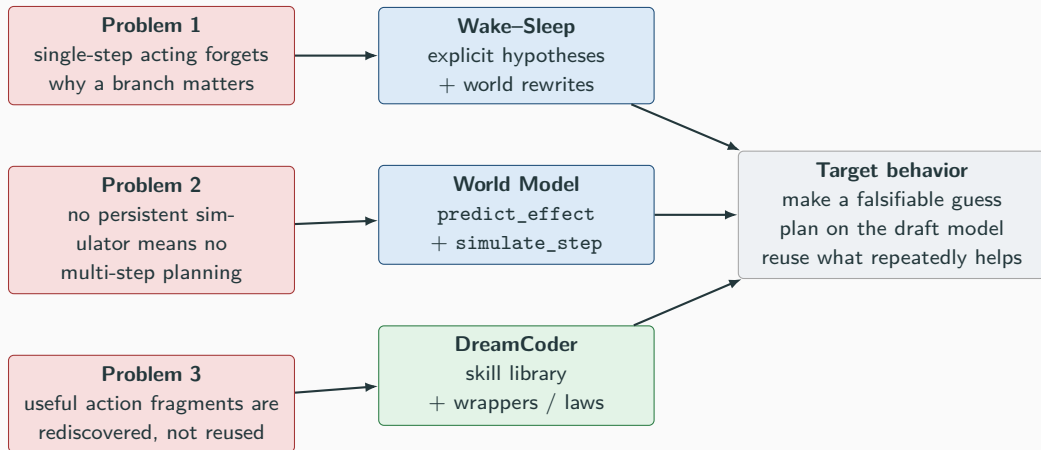


ARC-AGI-3 Symbolica Research

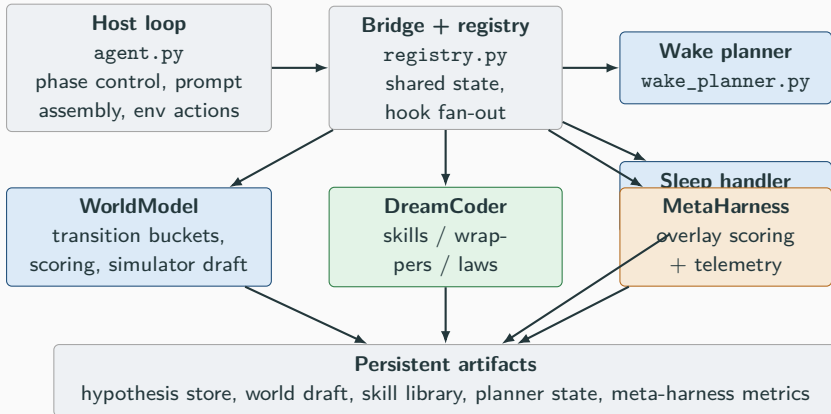
Runtime flow and current failure mode on 1s20

ARC-AGI-3 Symbolica Research

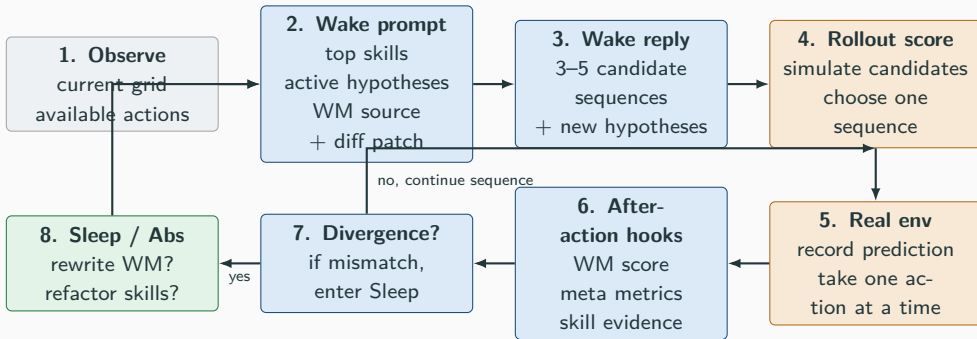
Why this framework was added



Runtime map in code



One online cycle



This is the active path in ArcgenticaResearch: online control is Wake → real env → Sleep, not pure MCTS.

What enters one Wake call

Input blocks from code

- current grid snapshot
- available action names
- top-3 retrieved skills
- active hypotheses from the frontier
- world-model source summary
- last divergence summary
- movement summary + changed cells
- initial-frame summary + amplified diff patch

Expected Wake output

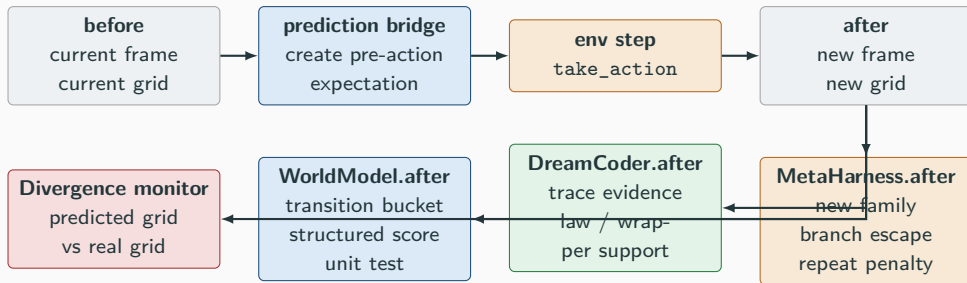
- candidate sequences
- rationale
- referenced skills
- 0–3 new falsifiable hypotheses

Why this matters

The prompt is no longer “what action next?”

It is “what sequence and what hypothesis should the runtime test next?”

What happens after one real action



The key fix in recent code was to restore this prediction bridge on the Rev M path so WorldModel can score expected vs actual transitions.

Online action selection vs optional MCTS

Actual online path now

- Wake LLM sees grid, hypotheses, skills, diff patch
- it proposes 3–5 action sequences
- runtime scores those sequences on the current world draft
- chosen sequence is executed in the real environment

Meaning

The online agent is already much closer to **“LLM writes the test sequence”** than to classical tree search.

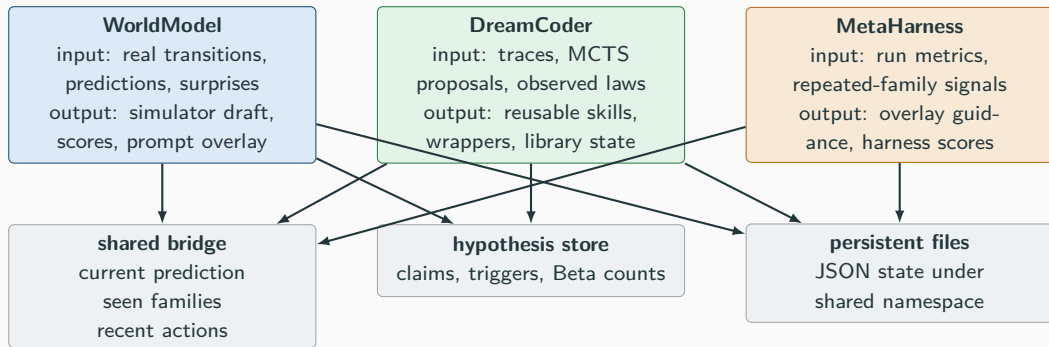
Optional MCTS side path

- lives in `planner.py`
- opens only when WM gate says the draft is good enough
- still uses UCB1 selection on the search tree
- may commit top paths into DreamCoder as skills

Meaning

UCB is still present, but it is **not** the main online control path.

Module inputs and outputs



Real hypothesis examples from the store

Example A: generic seed goal

Claim

[L0] A sequence of available actions reaches WIN.

Status

active, is_goal=True, support 0 / falsify 7

Why it is weak

It states the existence of a solve path, but says nothing about the actual mechanic.

Short check-code example

```
if action != 'ACTION4': result = None
else: compare before/after summaries and treat large remap as support
```

Example B: currently common mechanic

hypothesis

Claim

ACTION4 typically causes a large grid change.

Trigger

after ACTION4

Meaning

This is measurable and often confirmed, but it explains redraw magnitude, not goal progress.

Real world-model code example

Persisted predict_effect(action, observation) draft

```
def predict_effect(action, observation):  
    sig = observation.get('signature', '')  
    if sig == 'd2072020114e7cd91549' and action in ('ACTION2','ACTION4'):  
        return {  
            'expect_change': True,  
            'expected_diff_band': 'large',  
            'expected_next_grid': new_grid,  
            'latent_state_prediction': {...},  
            'goal_progress_prediction': {'progress_delta': 0.0}  
        }
```

What this tells us

The draft is executable code and already emits latent / goal fields, but the logic is still mostly empirical replay of observed deltas, not a compact mechanic simulator.

What changed from the naive world model

Before	Now	Meaning
one-step diff note	executable predict_effect draft	planning can call the simulator
free-form notes	explicit hypotheses + strict checks	failure becomes local and patchable
action retry loops	Wake sequence + Sleep rewrite + abstraction	search has memory across turns

Plan lens: intent, hypothesis, verification

Intent	Hypothesis	Verification
Wake should test mechanics	richer input yields better sequences + falsifiable hypotheses	candidate count, accepted hypotheses, startup stability
World model should learn mechanics	prediction bridge + Sleep rewrite activate empirical scoring	structured prediction count, scored matches, world update
Planner should matter only after WM quality rises	optional MCTS stays gated until the draft is useful	gate reason, planner state, MCTS vs BFS advantage
Skill library should store reusable control	DreamCoder promotes wrappers / controllers, not raw fragments	skill mix: wrappers, laws, executable leaves

Recent bounded results mapped to the plan

What the recent runs did prove

- Wake startup is stable again
- rate limiting is not the current blocker
- goal/state hypotheses survive early turns more often
- helper / whitelist mismatches were reduced

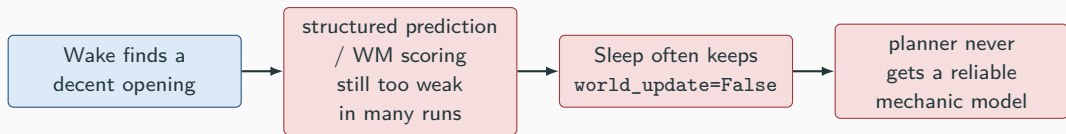
What the recent runs did *not* prove

- no level-up yet on 1s20
- no stable world-model rewrite handoff
- no reliable structured-prediction scoring loop yet
- MCTS is still mostly irrelevant online because the gate stays cold

Direct readout from reports

- `state_goal runs: startup_stalled=false, rate_limited=false`
- `worldscore_20260420a: startup_stalled=true`

Why 1s20 still stalls



So the runtime is better instrumented than before, but still fails at the exact handoff that should turn a good opening into a useful simulator rewrite.

Next repair loop

Intent

Make the online loop produce a scored world-model correction, not just better telemetry.

Hypothesis

If Rev M records structured predictions on the live action path, Sleep will finally receive evidence strong enough to rewrite the simulator draft.

Verification

- `startup_stalled = false`
- structured prediction count moves above zero
- empirical match/mismatch counts move
- `world_update = true` appears at least once